

INTERNAL ASSESSMENT

NAME	RATHIN BAGCHI
ROLL NO.:	2414517254
COURSE	BCA
SEMESTER	IV
SUBJECT	SYSTEM SOFTWARE – DCA2209

SET – I

Question 1

The 8086 microprocessor was 16-bit developed by Intel in 1978. It's internally split into two separate but independent functional units working simultaneously –

the Bus Interface Unit and the Execution Unit. This parallel operation is referred to as pipelining, and enables a processor to fetch the next instruction while it is currently executing the current one, thus boosting throughput over a processor that fetches and executes instructions one by one.

The Bus Interface Unit is in charge of all external bus operations. It serves as a link between the microprocessor and the outside world and coordinates communication with the memory and input/output devices. The BIU has a number of significant elements. The Segment Registers are four registers, each 16 bits wide, called Code Segment (CS), Data Segment (DS), Stack Segment (SS), and Extra Segment (ES). The 8086 has an address bus of 20 bits and can address 1 megabyte of memory, but has 16-bit internal registers. A physical address is obtained by shifting the value in the segment register to the left by four bits and adding an offset to the result, to give a 20-bit address.

The Instruction Pointer (or IP): The register that addresses the next instruction to be read from the code segment. The Instruction Queue is also part of the BIU, and it is a 6 byte FIFO buffer which holds the instructions requested for prefetch. When the Instruction queue is empty of two or more bytes, the BIU will automatically read more Instructions from memory and store them in the Instruction queue. The entire idea of pipelining in 8086 is based on this prefetching mechanism. The BIU contains Address Generation and Bus Control Logic to generate physical addresses, control read and write operations on the bus.

The Execution Unit is responsible for executing instructions. It does not fetch instructions from memory, but from the instruction queue which is maintained by the BIU. The EU includes the Arithmetic Logic Unit for performing all arithmetic functions, such as addition, subtraction, multiplication and division, as well as all logical functions such as AND, OR, NOT, and XOR. The EU also holds the Flag Register, which is a 16-

bit register on which each bit is a flag indicating the result of the previous operation.

Some of the flags that are important are the Zero flag, Sign flag, Carry flag, Overflow flag, and Parity flag. The flags are used by conditional jump instructions to direct the flow of the program.

The General Purpose Registers (GPR) in the EU are 16-bit, and can be split into two 8-bit halves; AX, BX, CX, and DX. The accumulator is used for arithmetic operations, labeled AX. BX is used for addressing purposes only. The count register in use for loop or shift operations is called CX. The data register used for input-output operations and multiplication is known as DX. Pointers and Index Registers such as SP, BP, SI and DI are also present within the EU for stack operations and memory addressing.

The BIU and EU run concurrently: The EU carries out the current instruction, while the BIU fetches the next instruction. When a branch instruction is executed by the EU, the queue is emptied and the EU starts fetching from the new address.

Question 2

In a single pass assembler, the source code is scanned only once by the assembler and the assembly language instructions are translated into machine language instructions in one pass. This method uses minimal memory and time, but has a fundamental problem, called the forward reference problem.

A forward reference is an instruction that references a symbol or label that is not defined at the place of occurrence in the single pass. For instance, a "jump" statement in the head of a program might jump to a label in the tail of the program. On the one pass the assembler sees this "jump" instruction, it must be able to locate the address of the label that it is jumping to in order to produce the appropriate machine code. At this point, however, the label has not yet been processed and its address is not defined in the source file, so the assembler does not know its address at this time.

This is easily corrected in a two-pass assembler since the first pass creates a symbol table containing all the addresses of labels, and the second pass resolves all references with this table. This is not possible with a single pass assembler and special techniques must be used to cope with undefined symbols.

The most popular method adopted by the single pass assembler is that it utilizes a table of incomplete instructions with a backpatching mechanism. If it sees a reference to a symbol it has not yet defined, the assembler outputs an incomplete or tentative machine instruction, with either an undefined address field, or a field that contains a placeholder value. It then records this incomplete instruction's location in a special data structure. At the same time it generates an entry in the symbol table for the undefined symbol, and sets up a chain of forward references between this entry and all the references to it.

Most often, this chain is used as a linked list. The symbol table for an undefined symbol will have an entry for each occurrence, with each entry pointing to the first place the symbol was used and a pointer to the next place the symbol was used, etc... This forms a chain of forward references, which are followed by all incomplete instructions waiting for the address of that symbol.

The assembler finally reaches the definition of the symbol (when it comes to the label in the source program) and fetches the symbol's true address and does backpatching.

Backpatching involves scanning the chain of forward references and placing the correct address in each forward reference instruction for this symbol. This is then stored as a defined symbol in the symbol table with the resolved address.

This is a "limited" two-

pass assembler approach: some single pass assemblers will make two passes over the source, with a pre-scan to collect labels and a subsequent scan to process them.

If there are undefined symbols that occur in the source code but are never defined, they are another matter.

Question 3a

A Macro Processor uses two key data structures during processing of macros in Assembly language programming: The Macro Name Table and Macro Definition Table. They're used together but for different purposes.

The Macro Name Table, MNT, is either an index or a directory of all the macros defined in the source program. An entry in the MNT consists of the name of the macro, and a pointer or index into the Macro Definition Table that provides a complete definition of the macro. The MNT is used during the expansion phase when the macro processor sees a macro call in the source code. This looks into the MNT and tries to find the name being called and, if it is found, looks for the corresponding definition in the MDT by using the pointer stored in the MNT.

Each macro definition contains the real definition that is stored on the Macro Definition Table (MDT). If a macro is defined in the source program with the MACRO and MEND directives, then all the statements in the macro body are stored sequentially in the MDT by the processor. The body of each macro is stored in a contiguous section of the MDT. The MDT is sort of containing the macro templates. When calling a macro during expansion, the processor fetches the related entries in the MDT, which are then processed and the formal parameters are replaced by actual parameters before the expanded code is inserted into the source stream.

To summarise the differences: MNT is a lookup table of macro names and pointers, it is small in size and used for lookup. The MDT is the real storage of macro bodies which are larger and used for retrieval and expansion. The MNT specifies that a macro exists and its location, while the MDT specifies the contents of a macro. These two, combined, provide for efficient macro definition and expansion in assembler macro processors.

Question 3b

An operating system or a large loader program is loaded into memory when the computer is initially turned on by a small and simple program called a bootstrap loader, which is stored in ROM. It is named after the expression "to pull oneself up by the

bootstraps": the computer requires a smaller, built-in program to load the larger program it needs to run.

On starting up the computer, or resetting it, the computer automatically fetches the instructions from a fixed address in the ROM memory of the computer and executes them. The first part of the bootstrap loader initializes the basic hardware and verifies that it is working correctly, such as making sure that the memory, CPU and basic input-output interfaces are all in order.

The next step in the bootstrap loader is to read the object code of the program to be executed from the secondary storage device (usually a disk) into main memory. It reads the object file sequentially, usually beginning with a header which gives information about the size of the program, its location when it is loaded into memory, and the location where it is executed.

The size information in the header is used by the loader to fetch the binary object code block, block by block or byte by byte from storage. No conversion from symbolic form is needed at this stage since object code is already in binary machine language format. One bit or word at a time is loaded into the memory at the correct address; this begins at the load address in the header of the program, and continues until the end of the program is reached.

After the entire object code is loaded into memory, the bootstrap loader jumps to the address given in the object file header for the beginning of the program and passes control to it. This will cause the processor to start fetching and executing instructions from that address, passing control from the bootstrap loader to the program loaded. The loaded program is usually a more advanced loader or the operating system kernel, that loads the entire operating system. This process of progressively stronger loaders is known as staged booting and is the basis for booting processes for all modern computers.

SET – II

Question 4

Relocation of a program: the ability to alter the address references in a program so that it may be properly loaded and run at a location other than the one at which it was originally assembled or compiled. The rationale for this is in the multiprogramming environment, the different programs do share memory and a program may not always be guaranteed to be in the same fixed memory location on each execution.

When a program is put together or compiled, it is translated as if it will be loaded into a specific memory address. This presumed starting address is known as the translated origin. This translated origin is used for all internal addresses within the program (jump targets, variable addresses, call addresses for subroutines etc.). If the program is made up with translated origin=0, then the address of a variable in the 10th byte of the program is 10 in the object code.

The linker or loader will, in fact, load the program at a different starting address than the address in the translated origin. This is the actual address in physical memory from which the program is to be loaded, and is known as the linked origin or load origin. Suppose that the translated origin is zero, but the program actually runs at memory location 5,000; then all absolute address references in the program will be incorrect by 5,000.

The amount of difference between the linked origin and the translated origin is known as the relocation factor. In the above example, the relocation factor is 5,000. The linker/loader must add this relocation factor to all address-sensitive locations in the program when relocating the program properly. Constant data values and immediate operands are not addresses and cannot be modified by a program. Thus, the linker relies on a relocation bit map or relocation dictionary that is part of the object file, to determine where there are relocatable addresses, and where there are absolute ones.

The linker fetches the value at each location marked as "relocatable" in the dictionary, adds the relocation factor, and stores the new address in the location. Once all of the relocatable items have been processed, each address in the program contains the proper memory address of the memory location of the code or data item at that address. This program can then be run without any problems, whichever memory location it is loaded into.

If the system consists of several separately compiled modules that are linked together, each module can have its own translated origin, and the linker will determine each module's position in the program and calculate individual relocation factors for each. This enables separately developed and compiled modules to be linked together into one executable binary, which executes as a single program.

Question 5

Peripheral Component Interconnect (PCI) is a standard for connecting various hardware components to a computer motherboard. It has some of the most critical features such as auto-configuration at boot time (Plug and Play). This saves the hassle of manually programming the addresses of memory, interrupt lines, and input-output port ranges.

The boot process will include a step called PCI bus enumeration carried out by the system firmware, which on older systems is usually BIOS and on newer systems is the UEFI. It systematically cycles through every possible combination of bus number, device number and function number, all possible slots on each PCI bus. It reads a group of registers in the configuration space of the device for each combination. All PCI devices must have a common 64 bytes or more configuration space that can be accessed by the device's firmware without any configuration. The first bytes in this space are the Vendor ID and Device ID that are used to uniquely identify the manufacturer and specific device. The firmware is able to read a valid Vendor ID, indicating that there is a device at that location.

When a device is detected, the firmware reads the Base Address Registers (BARs). Each BAR specifies what sort of resource the device requires and how much, in the form of a range of memory addresses or input-output port space. The size is determined by the firmware, which writes all ones to the BAR and then reads the value back. The amount of zero bits at the lowest end is the required size. The firmware then maps into an available space of addresses, selects a non-overlapping address range, and stores the address range in the BAR. It also configures interrupt lines in the interrupt routing table. Once all the devices are set up, each device will be able to identify its resources and is ready to use.

After the enumeration of the firmware, the device drivers really play a crucial role in the PCI initialization. The firmware doesn't know the meaning of the function each device is used for, only the assignment of resources. Once the operating system loads, it does an auto-enumeration of the PCI bus and fetches the Vendor ID and Device ID of each device found. This relies on these identifiers to locate and load the right device driver for each device.

The device driver is software developed by the device manufacturer which knows the internal registers, protocols and behaviour of that device. When loaded, the driver can read the base addresses the device needs to communicate with from the device's BARs, and knows exactly where to send the data. It then does device-specific initialisation, sets up operational modes, initialises interrupt handlers and prepares the device for use by higher level software and applications. If this hardware was not used by the operating system, there would have been no way to use it other than by the minimum configuration that the firmware performs. Firmware enumeration and driver-based initialization make up the full extent of the automatic configuration of the PCI that allows modern computers to transparently use different hardware.

Question 6a

Android is the most popular mobile operating system, and it's installed on billions of devices around the world. This ubiquity puts security at the heart of its design and development, making it an attractive target for attackers.

Android security architecture is based on layered architecture. It's based on the Linux kernel, which supports process isolation, user-based permissions and abstraction of the hardware. Every app application on an Android device is executed in a separate isolated process, each having its own Linux user ID. By default, one application does not have access to the memory, files or resources of any other application. This isolation is known as application sandbox, and is one of the most basic security features in Android OS.

The other important security layer in effect is the permission system. Each application must explicitly identify in its manifest file all of the sensitive capabilities it will be using, including camera, microphone, location, contacts and Internet. On newer version

s of Android, the user is asked to grant or refuse a particular permission when the application is run, not at install time, and can have a fine-grained control over what each app can do. This restricts damage that can be done by a malicious or compromised application.

When security is lacking, modern smartphone applications pose great threats. Financial applications, such as banking, are ones that possess highly sensitive information and could be the subject of a malware attack to steal information or present a fake login screen. Personal communications and identity info is stored in social media apps. Health apps have access to very personal information. Malware disguised as legitimate apps that are available in unofficial app stores, spyware that can access the camera and microphone without user knowledge, ransomware that encrypts device storage and adware that tracks user activity for unauthorized user profiling are among the threats that attackers can use against Android devices.

Android does that via multiple other methods. Google Play Protect's machine learning technology scans installed apps for malicious activity on an ongoing basis. The signing of the application guarantees that the application is being updated by the same developer who originally published the application, so that it is not tampered with. Verified Boot verifies the integrity of the operating system when it starts up, and can discover if there is unauthorized alterations of system partitions. Security patch updates are issued often to fix known security holes in the OS and Linux kernel.

Android's security model also features SELinux, Security Enhanced Linux, which is a mandatory access control policy that is enforced in the kernel, meaning that if a system process gets compromised, it cannot do as much as it could otherwise. In the event of a Lost or Stolen Device, device storage is encrypted to secure the data. The use of biometric authentication provides a robust access control measure.

Android still has its share of problems, however. Fragmentation is a phenomenon in which numerous devices have outdated versions with the lack of the latest security patches, as manufacturers and carriers do not distribute them quickly. People who install apps from third-party sources skip many of the protections provided by the Play Store. A social engineering attack is when users are tricked into giving too much permission to malicious software. Android security will continue to depend on ongoing cooperation among Google, device makers, application developers, and savvy users.

Question 6b

Simple service discovery protocol (SSDP) is a network protocol that enables the discovery of devices and services in a local network without having to set anything up manually or use a centralized directory server. It is a fundamental part of the Universal Plug and Play (UPnP) framework, which allows devices such as printers, smart TVs, routers and media servers to automatically broadcast their presence and ability to other devices on the local network.

SSDP is based on multicasting. The device broadcasts a multicast announcement message (SSDP ALIVE notification) to a well-

known multicast address when it joins a local network. This announcement and the discovery of the new device and its services are sent to all devices listening on that address on the local network. A device seeks a particular kind of service by sending an M-

SEARCH message. A device that provides the requested service returns its location and service description, which is usually a URL that points to an XML document detailing the service. Discovery is automatic, fast and doesn't require any prior knowledge of what devices are on the network. But, SSDP can only operate on the local network segment, and cannot operate on the Internet, as multicast packets cannot pass through routers between networks.

Another UDDI discovery mechanism, called Universal Description, Discovery, and Integration (UDDI), was developed for the business-to-

business discovery of web services on the internet. In a centralized model like UDDI, the idea is to have a "business directory" or "telephone book" for the Web, where organizations list information about the Web services they provide and other organizations on the web discover and use them.

There are three types of information in a UDDI registry. The contents of white pages include the basic information regarding a service provider like its name, address, and contact details. Yellow pages list businesses and services by category to find them by their category. Green pages have technical information about the services provided, such as the service endpoints, communication protocols and interface specification to actually invoke the service.

Organizations that want to have their services discovered register them with a UDDI registry by publishing three kinds of information. The businesses or developers make a specific type of service call to the UDDI registry by using the standard APIs, obtain the technical description of services that meet the criteria, and build the integration aspects of the remote service into their own applications.

The main distinction between SSDP and UDDI is that SSDP is narrower in scope, architecture and purpose. SSDP is decentralized, automatic and is intended for spontaneous network discovery between consumers such as smart home devices and printers without any previous setup and registration. UDDI is centralized, and it needs to be explicitly registered by service providers, and is intended for formal business-to-business integration over the internet, where services can be found by the partner business in another country. SSDP works in the network layer with multicast while UDDI works in the application layer via web-based query interfaces. They are at the two extremes of service discovery: informal/local and formal/global.